

Sequential Decision Making: Theory and Practice

Lecture 5: Online RL

Yuejie Chi

Yale Statistics & Data Science



Fall 2026

Outline

Linear function approximation and beyond

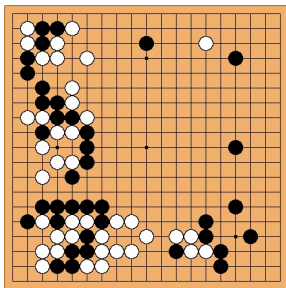
Online RL with ϵ -greedy exploration

Monte Carlo, Sarsa and Q-learning with GLIE

The deadly triad

Online RL with UCB-based exploration

Humungous state space



$$S \approx 2 \cdot 10^{170}$$

Exploiting **low-complexity model** is essential for efficient RL!

- Save computation/space/data
- Generalize across state or states and actions

Function approximation

Function approximation

The *object of interest* possesses some *low-dimensional representation*.

- value function
- Q-function
- transition kernel
- reward
- policy
- Parametric:
 - Linear combinations of features
 - Neural networks
- Nonparametric:
 - Decision trees
 - Nearest neighbors

Policy evaluation with linear function approximation

Linear V/Q function approximation

The **value/Q** function is a *linear* combination of features:

$$V(s; w) = \phi(s)^\top w,$$

$$Q(s, a; w) = \psi(s, a)^\top v,$$

where

- $\phi(s)$ maps the state space $\mathcal{S}$ to $\mathbb{R}^{d_1}$;
- $\psi(s, a)$ maps the state-action space $\mathcal{S} \times \mathcal{A}$ to $\mathbb{R}^{d_2}$;
- w, v are the *low-dimensional embeddings* we wish to learn.

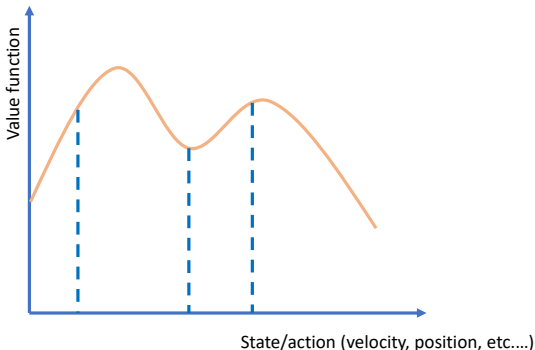
Typically, the number of features is much smaller than the dimension, i.e.

$$d_1 \ll |\mathcal{S}|, \quad d_2 \ll |\mathcal{S}||\mathcal{A}|.$$

We assume the feature maps are **known**.

Feature selection

When the states/actions are numbers, function approximation for value functions bears familiarity with that of **interpolation and regression** in supervised learning.



Example features: polynomials, Fourier basis, radial basis functions,...

Incremental update with **true** value

Objective function:

$$J(w) = \frac{1}{2} \mathbb{E}_{s \sim d^\pi} \left[\underbrace{(V^\pi(s) - V(s; w))^2}_{=: J(s; w)} \right] = \frac{1}{2} \mathbb{E}_{s \sim d^\pi} \left[(V^\pi(s) - \phi(s)^\top w)^2 \right],$$

which is *quadratic* w.r.t. w .

- Given access to $V^\pi(s)$, the stochastic gradient is evaluated as

$$\nabla_w J(s; w) = - \underbrace{(V^\pi(s) - \phi(s)^\top w)}_{\text{approx. error}} \phi(s).$$

- Update the weight w via

$$w \leftarrow w - \alpha \nabla_w J(s; w) = w + \alpha (V^\pi(s) - \phi(s)^\top w) \phi(s),$$

where α is the learning rate.

Incremental update with **target** value

However, in reality, we do not have access to $V^\pi(s)$ (otherwise we won't need to learn!).

Instead: replace $V^\pi(s)$ by its **target** $V_{\text{target}}^\pi(s)$ from MC or TD.

$$J(w) = \frac{1}{2} \mathbb{E}_{s \sim d^\pi} \left[\underbrace{\left(V_{\text{target}}^\pi(s) - \phi(s)^\top w \right)^2}_{=: J(s;w)} \right],$$

Update the weight w via

$$w \leftarrow w - \alpha \nabla_w J(s; w) = w + \alpha \left(V_{\text{target}}^\pi(s) - \phi(s)^\top w \right) \phi(s),$$

where α is the learning rate.

Examples of different targets

- In MC, use the return G_t

$$w \leftarrow w + \alpha (G_t - \phi(s)^\top w) \phi(s)$$

- In TD(0), use the TD target $r_t + \gamma V(s_{t+1}, w) = r_t + \gamma \phi(s_{t+1})^\top w$

$$w \leftarrow w + \alpha (r_t + \gamma \phi(s_{t+1})^\top w - \phi(s_t)^\top w) \phi(s_t)$$

- In TD(λ), use the λ -return G_t^λ

$$w \leftarrow w + \alpha (G_t^\lambda - \phi(s)^\top w) \phi(s)$$

These are “**semi-gradient**” methods, since we only consider the gradient of the function approximator, not the target.

Beyond linear function approximation

Objective function:

$$J(w) = \frac{1}{2} \mathbb{E}_{s \sim d^\pi} \left[\underbrace{(V^\pi(s) - V(s; w))^2}_{=: J(s; w)} \right],$$

where $V(s; w)$ is a differentiable function approximator.

- Given access to $V^\pi(s)$, the stochastic gradient is evaluated as

$$\nabla_w J(s; w) = \underbrace{(V^\pi(s) - V(s; w))}_{\text{approx. error}} \nabla_w V(s; w).$$

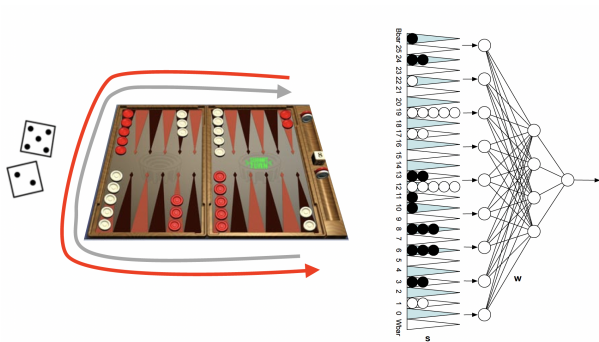
- Update the weight w via

$$w \leftarrow w - \alpha \nabla_w J(s; w) = w + \alpha (V^\pi(s) - V(s; w)) \nabla_w V(s; w),$$

where α is the learning rate.

TD-Gammon

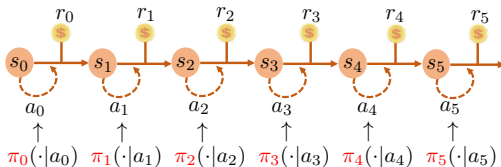
— [Tesauro, 1995]



- Value network: three-layer neural network
- Self-play: millions of games played against itself
- Beat the best human player of backgammon at the time

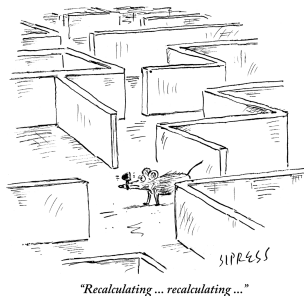
Online RL with ϵ -greedy exploration

Online RL



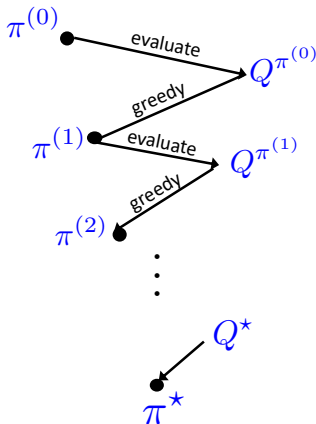
Exploration under an adaptive policy:

- trial-and-error
- sequential and online
- using samples



Model-free policy iteration?

Policy iteration



Can we run policy iteration using samples?

- We can perform the step of policy evaluation via Monte Carlo or TD-learning.
- However, when $\pi^{(t)}$ is deterministic, can only evaluate $Q^{\pi^{(t)}}(s, a)$ for $a = \pi^{(t)}(s)$ following policy $\pi^{(t)}$.
- Need to perform exploration!

ϵ -greedy exploration

- With probability $1 - \epsilon$ choose the greedy action
- With probability ϵ choose an action at random

$$\pi(a|s) = \begin{cases} \epsilon/|\mathcal{A}| + 1 - \epsilon & \text{if } a^* = \arg \max_{a \in \mathcal{A}} Q(s, a) \\ \epsilon/|\mathcal{A}| & \text{otherwise} \end{cases}$$

Theorem 1

For any ϵ -greedy policy π , the ϵ -greedy policy π' with respect to Q^π is an improvement, i.e. $V^{\pi'}(s) \geq V^\pi(s)$, $\forall s \in \mathcal{S}$.

- assumes exact policy evaluation

Proof of greedy policy improvement

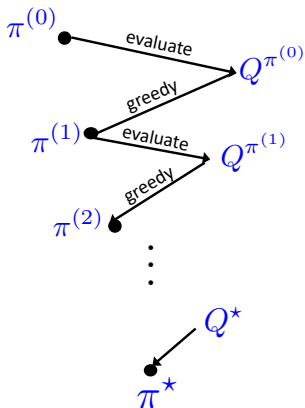
Proof: Let $m = |\mathcal{A}|$.

$$\begin{aligned} Q^\pi(s, \pi'(s)) &= \sum_{a \in \mathcal{A}} \pi'(a|s) Q^\pi(s, a) \\ &= \frac{\epsilon}{m} \sum_{a \in \mathcal{A}} Q^\pi(s, a) + (1 - \epsilon) \max_{a \in \mathcal{A}} Q^\pi(s, a) \\ &\geq \frac{\epsilon}{m} \sum_{a \in \mathcal{A}} Q^\pi(s, a) + (1 - \epsilon) \sum_{a \in \mathcal{A}} \frac{\pi(a|s) - \epsilon/m}{1 - \epsilon} Q^\pi(s, a) \\ &= \sum_{a \in \mathcal{A}} \pi(a|s) Q^\pi(s, a) = V^\pi(s). \end{aligned}$$

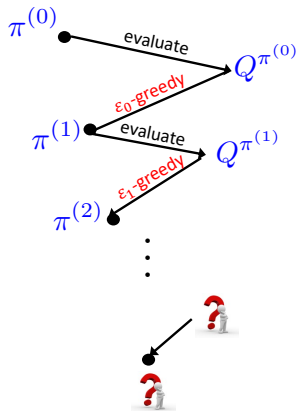
Therefore, by the policy improvement lemma, $V^{\pi'}(s) \geq V^\pi(s)$.

Generalized policy iteration

Policy iteration



Greedy policy iteration



Does greedy policy iteration converge to the optimal Q-function/policy, particularly when using samples?

Monte Carlo, Sarsa and Q-learning with GLIE

Greedy in the Limit with Infinite Exploration (GLIE)

Definition 2 (GLIE)

- **Infinite exploration.** All state-action pairs are explored infinitely many times,

$$\lim_{k \rightarrow \infty} N_k(s, a) = \infty$$

- **Greedy in the limit.** The policy converges on a greedy policy,

$$\lim_{k \rightarrow \infty} \pi_k(a|s) = \mathbb{I}(a = \operatorname{argmax}_{a' \in \mathcal{A}} Q_k(s, a'))$$

- ϵ -greedy is GLIE with $\epsilon_k = 1/k$.
- Boltzmann exploration is GLIE: $\pi_k(a|s) \propto e^{\beta_k(s)Q_k(s,a)}$ for appropriate choice of $\beta_k(s)$ [Singh et al., 2000].

GLIE Monte-Carlo control

Monte-Carlo control requires episodic environment (since it works only with complete sequences).

- 1 Sample a new episode using π ;
- 2 Update the Q-estimate using Monte-Carlo;
- 3 Update policy π using Q , e.g., ε -greedy.

Theorem 3

GLIE Monte-Carlo control converges to the optimal Q-function:

$$Q(s, a) \rightarrow Q^*(s, a).$$

Using bootstrapping for policy evaluation

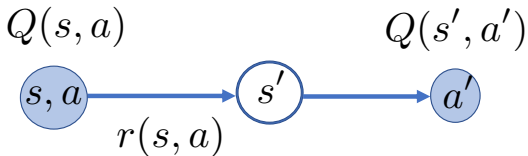
TD has several advantages over Monte-Carlo (MC):

- Lower variance
- Online
- Incomplete sequences

Natural idea: use TD instead of MC for policy evaluation

- apply TD to $Q(s, a)$
- use ϵ -greedy policy improvement
- update every time step (we don't need to wait for TD to converge)

Sarsa for on-policy Q-update



On-policy Q-update:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left(\underbrace{r(s, a) + \gamma Q(s', a')}_{\text{target}} - Q(s, a) \right)$$

- The name comes from updating using a 5-tuple: (s, a, r, s', a') .
- The method is an on-policy method because it tries to learn about policy π from experience sampled from π .
- Can combine TD(λ) in Sarsa for the target part.

Sarsa for on-policy control

① **Initial phase:** initialization $Q(s, a)$ arbitrarily. Set $Q(s, \cdot) = 0$ if s is a terminal state. Initial state s and initial policy π (e.g., uniform).

② For each round $t = 1, 2, \dots$:

- Choose $a \sim \pi(\cdot|s)$ for the current state s ;
- Take action a , observe $r = r(s, a)$ and next state s' ;
- Choose $a' \sim \pi(\cdot|s')$ from the next state s' and update the Q-value

$$Q(s, a) \leftarrow Q(s, a) + \alpha_t (r(s, a) + \gamma Q(s', a') - Q(s, a))$$

- $s \leftarrow s', a \leftarrow a'$.
- Update policy π using Q , e.g., ϵ -greedy.

Repeat this for every episode if in an episodic environment.

SARSA with linear function approximation

- Approximate the *on-policy* Q-function with

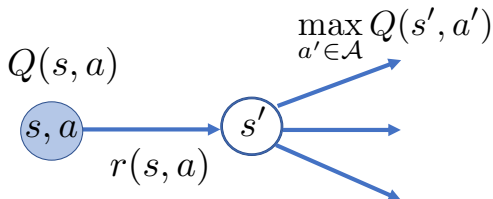
$$Q(s, a; w) = \psi(s, a)^\top v,$$

- Policy evaluation:** apply TD(0) to update the weight

$$v_{t+1} \leftarrow v_t + \alpha \left(r_t + \gamma \psi(s_{t+1}, a_{t+1})^\top v_t - \psi(s_t, a_t)^\top v_t \right) \psi(s_t, a_t)$$

- Policy improvement:** ϵ -greedy policy improvement.

Q-learning for off-policy Q-update



Off-policy Q-update:

$$Q(s, a) \leftarrow Q(s, a) + \alpha_t \left(\underbrace{r(s, a) + \gamma \max_{a' \in \mathcal{A}} Q(s', a')}_{\text{target}} - Q(s, a) \right)$$

- The method is an off-policy method because it tries to learn about an improved policy from experience sampled from π .

Q-learning for off-policy control

- ① **Initial phase:** initialization $Q(s, a)$ arbitrarily. Set $Q(s, \cdot) = 0$ if s is a terminal state. Initial state s and initial policy π (e.g., uniform).
- ② For each round $t = 1, 2, \dots$:
 - Choose $a \sim \pi(\cdot|s)$ for the current state s ;
 - Take action a , observe $r = r(s, a)$ and next state s' ;
 - **Update the Q-value**

$$Q(s, a) \leftarrow Q(s, a) + \alpha_t \left(r(s, a) + \gamma \max_{a' \in \mathcal{A}} Q(s', a') - Q(s, a) \right)$$

- $s \leftarrow s'$.
- Update policy π using Q , e.g., ϵ -greedy.

Repeat this for every episode if in an episodic environment.

Q-learning with linear function approximation

- Approximate the *off-policy* Q-function with

$$Q(s, a; w) = \psi(s, a)^\top v,$$

- **Policy evaluation:** using *Q-learning target* to update the weight

$$v_{t+1} \leftarrow v_t + \alpha \left(r_t + \gamma \max_a \psi(s_{t+1}, a)^\top v_t - \psi(s_t, a_t)^\top v_t \right) \psi(s_t, a_t)$$

- **Policy improvement:** ϵ -greedy policy improvement

Convergence of Sarsa/Q-learning with GLIE

Theorem 4 ([Singh et al., 2000])

Sarsa/Q-learning converges to the optimal action-value function, $Q(s, a) \rightarrow Q^(s, a)$, under the following conditions:*

- *GLIE sequence of policies $\pi_t(a|s)$;*
- *Robbins-Monro sequence of learning-rates*

$$\sum_{t=1}^{\infty} \alpha_t = \infty, \quad \sum_{t=1}^{\infty} \alpha_t^2 < \infty$$

- Due to GLIE, the policy will also converge to the optimal policy.

Case study: Q-learning versus Sarsa

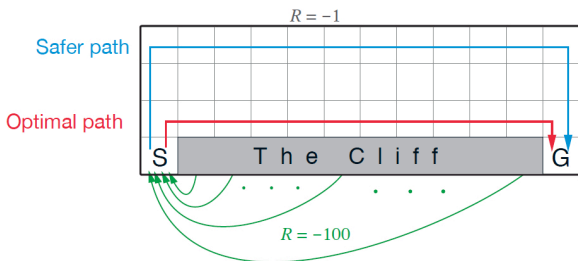


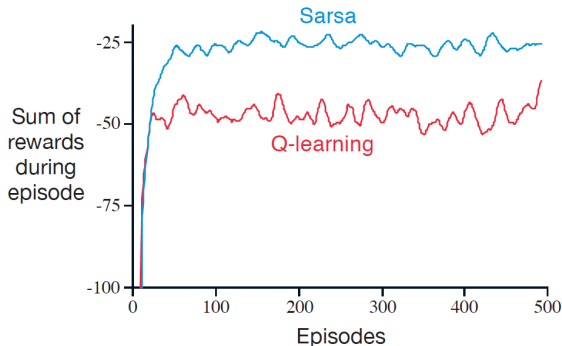
Figure credit: Sutton and Barto

Cliff walking:

- undiscounted, episodic task, with start and goal states, and the usual actions causing movement up, down, right, and left.
- Reward is -1 on all transitions except "The Cliff". Stepping into this region incurs a reward of -100 and sends the agent instantly back to the start.

Case study: Q-learning versus Sarsa

Both are executed with ϵ -greedy policy, $\epsilon = 0.1$.



- Q-learning learns the optimal policy but results in its occasionally falling off the cliff because of the “ ϵ -greedy” action selection.
- Sarsa takes the action selection into account and learns the longer but safer path through the upper part of the grid.

Double Q-learning with GLIE

- ① **Initial phase:** initialization Q_1, Q_2 arbitrarily. Initial state s and initial policy π (e.g., uniform).
- ② For each round $t = 1, 2, \dots$:
 - Choose $a \sim \pi(\cdot|s)$ for the current state s ;
 - Take action a , observe $r = r(s, a)$ and next state s' ;
 - Update the Q-value using one of the following randomly

$$Q^1(s, a) \leftarrow Q^1(s, a) + \alpha_t \left(r(s, a) + \gamma Q^2(s, \underset{a \in \mathcal{A}}{\operatorname{argmax}} Q^1(s', a)) - Q^1(s, a) \right),$$

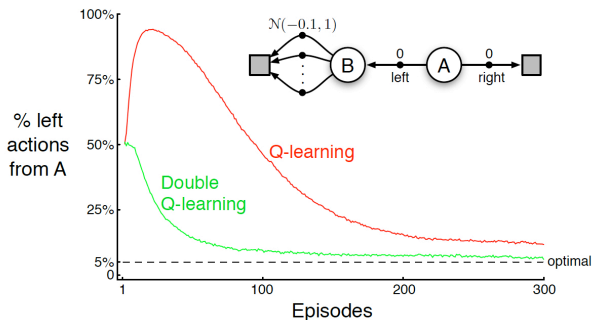
or

$$Q^2(s, a) \leftarrow Q^2(s, a) + \alpha_t \left(r(s, a) + \gamma Q^1(s, \underset{a \in \mathcal{A}}{\operatorname{argmax}} Q^2(s', a)) - Q^2(s, a) \right).$$

- $s \leftarrow s'$.
- Update policy π using $(Q^1 + Q^2)/2$, e.g., ε -greedy.

Repeat this for every episode if in an episodic environment.

Mitigating overestimation via double Q-learning



- Q-learning initially learns to take the left action much more often than the right action, and always takes it significantly more often than the 5% minimum probability enforced by ϵ -greedy action selection with $\epsilon = 0.1$.
- Double Q-learning is essentially unaffected by the overestimation bias.

The deadly triad

Off-policy evaluation

Sometimes we are interested in evaluating policy π different from behavior policy μ .

- Learn from observing humans or other agents
- Re-use experience generated from old policies
- Learn about optimal policy while following exploratory policy
- Learn about multiple policies while following one policy



Can we adapt our ideas so far to off-policy evaluation?

Importance sampling

Evaluate an expectation using samples from another distribution. Assume $Q(x) > 0$ whenever $P(x) > 0$.

$$\begin{aligned}\mathbb{E}_{X \sim P}[f(X)] &= \sum P(X)f(X) \\ &= \sum Q(X) \frac{P(X)}{Q(X)} f(X) \\ &= \mathbb{E}_{X \sim Q} \left[\frac{P(X)}{Q(X)} f(X) \right]\end{aligned}$$

- The importance weights: $\frac{P(X)}{Q(X)}$
- For policy evaluation, P and Q describe trajectories under π and μ .

Importance sampling for off-policy Monte Carlo

Multiply corrections through the last action before terminal state s_T :

$$G_t^{\pi/\mu} = \frac{\pi(a_t|s_t)}{\mu(a_t|s_t)} \frac{\pi(a_{t+1}|s_{t+1})}{\mu(a_{t+1}|s_{t+1})} \dots \frac{\pi(a_{T-1}|s_{T-1})}{\mu(a_{T-1}|s_{T-1})} G_t$$

Update value towards **corrected** return:

$$V(s_t) \leftarrow V(s_t) + \alpha \left(G_t^{\pi/\mu} - V(s_t) \right)$$

- High variance when $\frac{\pi(a|s)}{\mu(a|s)}$ is large
- Requires $\mu(a|s) > 0$ whenever $\pi(a|s) > 0$ on target-policy trajectories.

Importance sampling for off-policy TD

- Weight TD target by importance sampling [Precup et al., 2001]

$$V(s_t) \leftarrow V(s_t) + \alpha \left(\frac{\pi(a_t|s_t)}{\mu(a_t|s_t)} (r_t + \gamma V(s_{t+1})) - V(s_t) \right)$$

- Uses a single-step ratio, which can reduce variance relative to full-episode weighting.

Off-policy TD(0) with function approximation

$$J(w) = \frac{1}{2} \mathbb{E}_{s \sim d^{\pi}} \left[\underbrace{(V^{\pi}(s) - V(s; w))^2}_{=: J(s; w)} \right] = \frac{1}{2} \mathbb{E}_{s \sim d^{\pi}} \left[(V^{\pi}(s) - \phi(s)^{\top} w)^2 \right].$$

- Using the TD target $r_t + \gamma V(s_{t+1}, w) = r_t + \gamma \phi(s_{t+1})^{\top} w$, the semi-gradient is evaluated as

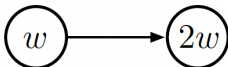
$$\nabla_w J(s_t; w) = - \underbrace{\left(r_t + \gamma \phi(s_{t+1})^{\top} w - \phi(s_t)^{\top} w \right)}_{\text{TD error } \delta_t} \phi(s_t).$$

- Update the weight w via

$$w_{t+1} = w_t - \alpha_t \underbrace{\frac{\pi(a_t|s_t)}{\pi_b(a_t|s_t)}}_{=: \rho_t} \nabla_w J(s_t; w) = w_t + \alpha_t \rho_t \delta_t \phi(s_t).$$

Off-policy TD(0) might diverge

Intuition: only one action is available, and it results deterministically in a transition to the second state with a reward of 0 [Sutton and Barto, 2018]:



- The linear function approximation assumes the value takes the form

$$[w, 2w] \quad \text{with} \quad \phi(\text{left}) = 1, \phi(\text{right}) = 2.$$

- For one transition from left state to right state, we have

$$\begin{aligned} \delta_t &= r_t + \gamma V(\text{right}) - V(\text{left}) = \gamma 2w_t - w_t = (2\gamma - 1)w_t, \\ \rho_t &= 1. \end{aligned}$$

- The off-policy TD(0) updates

$$w_{t+1} = w_t + \alpha_t \rho_t \delta_t \phi(\text{left}) = (1 + \alpha_t(2\gamma - 1)) w_t.$$

Diverges whenever $\gamma > 1/2$ for any $\alpha_t > 0$ if we do this over and over!

Baird's example

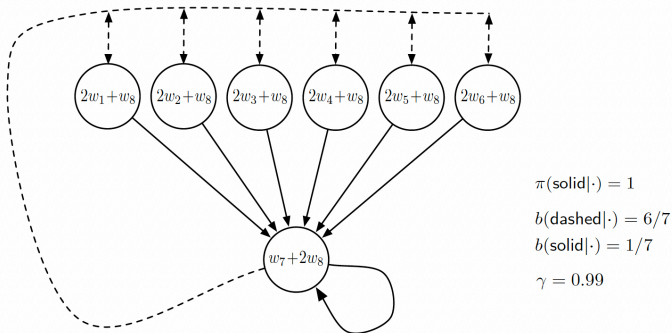


Figure 11.1: Baird's counterexample. The approximate state-value function for this Markov process is of the form shown by the linear expressions inside each state. The **solid** action usually results in the seventh state, and the **dashed** action usually results in one of the other six states, each with equal probability. The reward is always zero.

Figure source: [Sutton and Barto, 2018]

Baird's example explained

- 7 states, feature dimension = 8!!!
- The set of features is linearly independent, e.g.

$$\phi(1) = [2, 0, 0, 0, 0, 0, 0, 1]^\top$$

- The true value function is

$$V^\pi(s) = 0, \quad \text{which can be exactly approximated by } w = 0.$$

- The behavior policy π_b offers a path to skip the absorbing state 8 of π , creating a path mimicking our intuition earlier (focusing on w_8).
- We will be okay with on-policy evaluation.

Numerical divergence on Baird's example

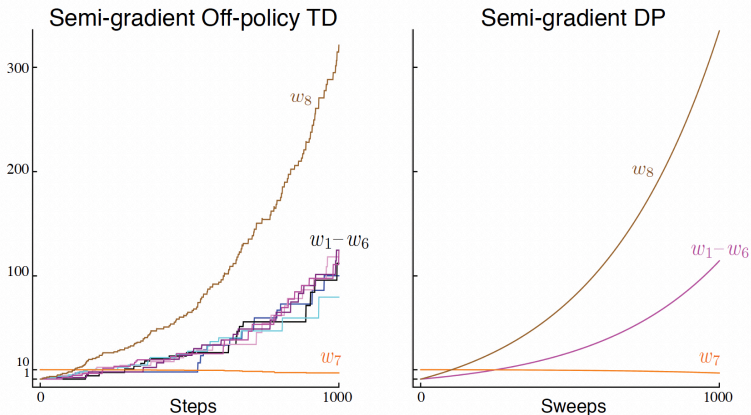


Figure 11.2: Demonstration of instability on Baird's counterexample. Shown are the evolution of the components of the parameter vector $\mathbf{w}$ of the two semi-gradient algorithms. The step size was $\alpha = 0.01$, and the initial weights were $\mathbf{w} = (1, 1, 1, 1, 1, 1, 1, 10, 1)^T$.

Figure source: [Sutton and Barto, 2018]

The deadly triad



Richard Sutton

The risk of divergence arises whenever we combine:

- **Function approximation:**
significantly generalizing from large numbers of examples
- **Bootstrapping:**
learning value estimates from other value estimates, as in dynamic programming and temporal-difference learning
- **Off-policy learning:**
learning about a policy from data not due to that policy, as in Q-learning, where we learn about the greedy policy from data with a necessarily more exploratory policy

Any two without the third is okay.

Possible remedies

- More careful algorithm designs [Sutton et al., 2009]:
 - Gradient TD (GTD)
 - TD with gradient correction (TDC)
 - Emphatic TD [Sutton et al., 2016], etc...
- Using a target network [Mnih et al., 2015, Zhang et al., 2021]:

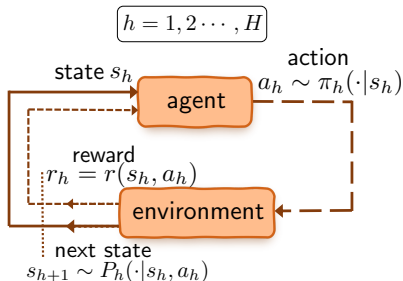
$$f(s_t, a_t; v) = \frac{1}{2} \left(r_t + \gamma \max_a Q_{\text{target}}(s_{t+1}, a; v) - Q(s_t, a_t; v) \right)^2$$

- Target network Q_{target} : periodically synced by the value network.
- Value network Q : updated via gradient methods.

A key ingredients in (*double*) deep *Q-learning* (DQN).

Online RL with UCB-based exploration

Finite-horizon nonstationary MDPs

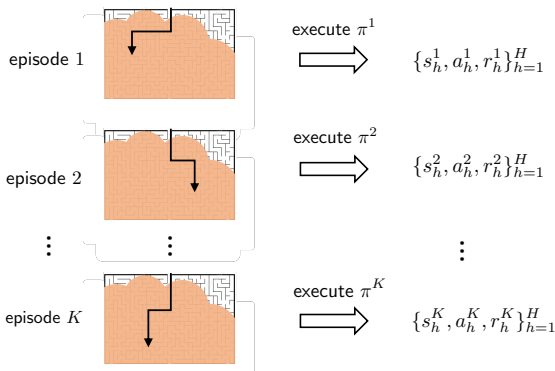


- H : horizon length
- $\mathcal{S}$: state space with size S
- $\mathcal{A}$: action space with size A
- $r_h(s_h, a_h) \in [0, 1]$: immediate reward in step h
- $\pi = \{\pi_h\}_{h=1}^H$: policy (or action selection rule)
- $P_h(\cdot | s, a)$: transition probabilities in step h

Online RL: interacting with real environments

Sequentially execute MDP for K episodes, each consisting of H steps

— *sample size: $T = KH$*



exploration (exploring unknowns) vs. **exploitation** (exploiting learned info)

Regret lower bounds

Theorem 5 ([Domingues et al., 2021])

For any algorithm, there exists an episodic MDP $\mathcal{M}_\pi$ whose transitions depend on the stage h , such that for $T \geq H^2SA$,

$$\mathbb{E}[\text{Regret}(T)] \geq \frac{1}{48\sqrt{6}} \sqrt{H^2SAT}.$$

- Ignoring other factors, the regret is at least

$$\Omega(\sqrt{T}).$$

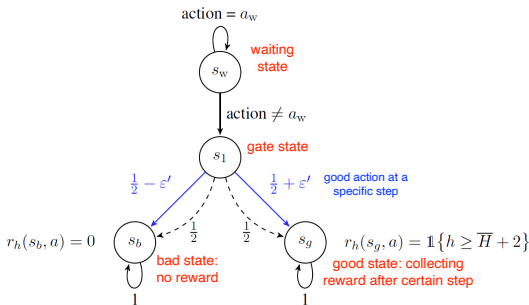
- The bound also reflects the impact of horizon length H and size of the state-action space SA . Note that the value function is on the order of H , so the “normalized” regret scales as

$$\frac{\mathbb{E}[\text{Regret}(T)]}{H} \gtrsim \sqrt{SAT} = \sqrt{SAHK}.$$

Construction of hard MDP

- Recall that the regret lower bound for an n -arm bandit (with normalized reward) is $\Omega(\sqrt{nT})$.
- It amounts to find a hard MDP that operates like a HSA -arm bandit (with reward $\sim H$).

Illustration of the hard MDP when $S = 4$. Taking $\bar{H} = \Theta(H)$.

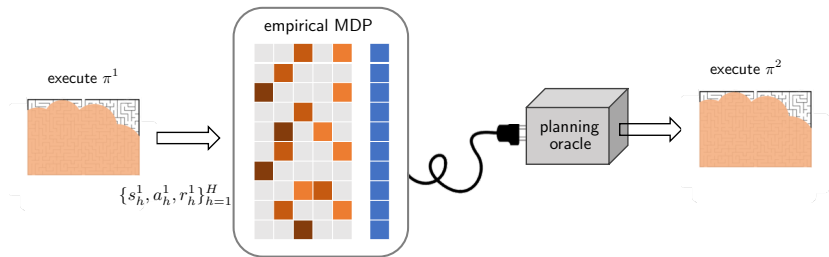


— Figure credit: [Domingues et al., 2021]

Can we design algorithms that achieve near-optimal regret?

Model-based RL with UCB exploration

Online RL with model-based approach



- Use **all** the previous data to estimate transitions (empirical frequencies)
- Apply planning (e.g., value iteration) on the estimated model to learn an updated policy for the next episode

How to balance exploration and exploitation in this framework?

UCB-VI: ideas

Motivated by the bandit UCB algorithm, [Azar et al., 2017] introduced **upper confidence bound (UCB)** into value iteration (VI).

- **Original VI:** Backtrack $h = H, H - 1, \dots, 1$:

$$Q_h(s, a) \leftarrow \underbrace{r_h(s_h, a_h)}_{\text{immediate reward}} + \underbrace{\hat{P}_{h,s,a} V_{h+1}}_{\text{next step's value}},$$
$$V_h(s) \leftarrow \max_{a \in \mathcal{A}} Q_h(s, a),$$

where $\mathbb{E}_{s' \sim P_h(\cdot | s, a)} V_{h+1}(s') = P_{h,s,a} V_{h+1}$ and $\hat{P}_{h,s,a}$ is the empirical estimate of $P_{h,s,a}$.

- Exploitation, but no exploration.
- Adding the UCB to $Q_h(s, a)$ similar to the bandit UCB algorithm.

UCB-VI: uncertainty quantification

Uncertainty in the next-step value $\hat{P}_{h,s,a}V_{h+1}$: recall that by Hoeffding's inequality and union bound, with probability at least $1 - \delta$,

$$\left\| (\hat{P}_{h,s,a} - P_{h,s,a})V_{h+1}^* \right\|_{\infty} \lesssim \sqrt{\frac{H^2 \iota}{N_h(s,a)}},$$

where $N_h(s,a)$ is number of visits in (s,a) at step h .

Optimistic VI: run VI using rewards $\{r_h(s_h, a_h) + b_h(s_h, a_h)\}$

$$Q_h(s,a) \leftarrow \min \left\{ H - h + 1, \underbrace{r_h(s_h, a_h)}_{\text{immediate reward}} + \underbrace{\hat{P}_{h,s,a}V_{h+1}}_{\text{next step's value}} + \underbrace{b_h(s_h, a_h)}_{\text{bonus}} \right\},$$

$$V_h(s) \leftarrow \max_{a \in \mathcal{A}} Q_h(s,a),$$

where the bonus is $b_h(s_h, a_h) \asymp \sqrt{\frac{H^2 \iota}{N_h(s,a)}}$.

UCB-VI: algorithm

For each episode k :

- ① Backtrack $h = H, H - 1, \dots, 1$: run **optimistic value iteration**

$$Q_h(s, a) \leftarrow \min \left\{ H - h + 1, \underbrace{r_h(s_h, a_h)}_{\text{immediate reward}} + \underbrace{\hat{P}_{h,s,a} V_{h+1}}_{\text{next step's value}} + \underbrace{b_h(s_h, a_h)}_{\text{bonus}} \right\},$$

$$V_h(s) \leftarrow \max_{a \in \mathcal{A}} Q_h(s, a),$$

- ② Forward $h = 1, \dots, H$: take action according to the greedy policy

$$\pi_h(s) \leftarrow \operatorname{argmax}_{a \in \mathcal{A}} Q_h(s, a)$$

and collect $\{s_h, a_h, r_h\}_{h=1}^H$.

Optimism in the face of uncertainty

Lemma 6 (Optimism)

With probability at least $1 - \delta$, it follows

$$Q_h^k(s, a) \geq Q_h^*(s, a), \quad V_h^k(s) \geq V_h^*(s)$$

for all (k, h, s, a) .

Optimism in the face of uncertainty:

acting according to $Q_h^k(s, a)$, which is an upper bound of the true $Q_h^*(s, a)$.



Regret bound of UCB-VI with Hoeffding bonus

Theorem 7 ([Azar et al., 2017])

Let $\delta \in (0, 1)$. With probability at least $1 - \delta$, the regret of UCB-VI with Hoeffding bonus satisfies

$$\text{Regret}(T) \lesssim \sqrt{H^3 S A T \iota} + H^3 S^2 A \iota^3,$$

where $\iota = \log(H S A T / \delta)$.

- The regret bound scales as

$$\sqrt{H^3 S A T} \quad \text{as soon as} \quad T \gtrsim \underbrace{H^3 S^3 A}_{\text{burn-in cost}}.$$

which is sub-optimal by a factor of $\sqrt{H}$.

- By the optimism principle, the regret is bounded by

$$\text{Regret}(T) = \sum_{k=1}^K \left(V_1^*(s_1^k) - V_1^{\pi^k}(s_1^k) \right) \leq \sum_{k=1}^K \left(V_1^k(s_1^k) - V_1^{\pi^k}(s_1^k) \right).$$

Tighter UCB leads to smaller regret.

Q-learning with UCB exploration

UCB-Q [Jin et al., 2018] modifies classical Q-learning with exploration bonus: at the transition (s_h, a_h, s_{h+1})

$$Q_h(s_h, a_h) \leftarrow \underbrace{(1 - \alpha_t)Q_h(s_h, a_h) + \alpha_t (r(s_h, a_h) + V_{h+1}(s_{h+1}))}_{\text{classical Q-learning}} + \underbrace{\alpha_t b_h(s_h, a_h)}_{\text{bonus}}$$

- Using Hoeffding-type bonus to ensure the optimism property:

$$b_h(s, a) \asymp \sqrt{\frac{H^3 \epsilon}{N_h(s, a)}}$$

Large variability in stochastic update rules.

- Rescaled linear learning rates:

$$\alpha_t = \frac{H+1}{H+t}, \quad t = N_h(s, a)$$

UCB-Q: algorithm with Hoeffding bonus

For each episode k :

① For $h = 1, \dots, H$:

- ① Take action according to the greedy policy $\pi_h(s) = \operatorname{argmax}_{a \in \mathcal{A}} Q_h(s, a)$ and observe s_{h+1} ;
- ② Update the count $t = N_h(s_h, a_h) \leftarrow N_h(s_h, a_h) + 1$;
- ③ Compute the bonus $b_h(s_h, a_h)$;
- ④ Update the visited entry of Q -function:

$$Q_h(s_h, a_h) \leftarrow \underbrace{(1 - \alpha_t)Q_h(s_h, a_h) + \alpha_t (r(s_h, a_h) + V_{h+1}(s_{h+1}))}_{\text{classical Q-learning}} + \underbrace{\alpha_t b_h(s_h, a_h)}_{\text{bonus}}$$

⑤ Update value function:

$$V_h(s_h) \leftarrow \min\{H - h + 1, \max_a Q_h(s_h, a)\}.$$

Regret bound of UCB-Q with Hoeffding bonus

Theorem 8 ([Jin et al., 2018])

Let $\delta \in (0, 1)$. With probability at least $1 - \delta$, the regret of UCB-Q with Hoeffding bonus satisfies

$$\text{Regret}(T) \lesssim \sqrt{H^4 SAT\iota},$$

where $\iota = \log(HSAT/\delta)$.

- The regret bound

$$\sqrt{H^4 SAT}$$

is sub-optimal by a factor of H . No burn-in cost!

- Can be improved to $\sqrt{H^3 SAT}$ by using variance-aware concentration bounds (i.e., Bernstein inequality) to construct the UCB.

References I



Azar, M. G., Osband, I., and Munos, R. (2017).

Minimax regret bounds for reinforcement learning.

In *Proceedings of the 34th International Conference on Machine Learning*, pages 263–272. JMLR.org.



Bradtke, S. J. and Barto, A. G. (1996).

Linear least-squares algorithms for temporal difference learning.

Machine learning, 22(1-3):33–57.



Domingues, O. D., Ménard, P., Kaufmann, E., and Valko, M. (2021).

Episodic reinforcement learning in finite MDPs: Minimax lower bounds revisited.

In *Algorithmic Learning Theory*, pages 578–598. PMLR.



Jin, C., Allen-Zhu, Z., Bubeck, S., and Jordan, M. I. (2018).

Is Q-learning provably efficient?

In *Advances in Neural Information Processing Systems*, pages 4863–4873.



Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., Graves, A., Riedmiller, M., Fidjeland, A. K., Ostrovski, G., et al. (2015).

Human-level control through deep reinforcement learning.

Nature, 518(7540):529–533.



Precup, D., Sutton, R. S., and Dasgupta, S. (2001).

Off-policy temporal-difference learning with function approximation.

In *ICML*, pages 417–424.

References II



Singh, S., Jaakkola, T., Littman, M. L., and Szepesvári, C. (2000).
Convergence results for single-step on-policy reinforcement-learning algorithms.
Machine learning, 38:287–308.



Sutton, R. S. and Barto, A. G. (2018).
Reinforcement learning: An introduction.
MIT press.



Sutton, R. S., Maei, H. R., Precup, D., Bhatnagar, S., Silver, D., Szepesvári, C., and Wiewiora, E. (2009).
Fast gradient-descent methods for temporal-difference learning with linear function approximation.

In Proceedings of the 26th annual international conference on machine learning, pages 993–1000.



Sutton, R. S., Mahmood, A. R., and White, M. (2016).
An emphatic approach to the problem of off-policy temporal-difference learning.
The Journal of Machine Learning Research, 17(1):2603–2631.



Tesauro, G. (1995).
Temporal difference learning and td-gammon.
Communications of the ACM, 38(3):58–68.



Tsitsiklis, J. and Van Roy, B. (1997).
An analysis of temporal-difference learning with function approximation.
IEEE Transactions on Automatic Control, 42(5):674–690.

References III



Zhang, S., Yao, H., and Whiteson, S. (2021).

Breaking the deadly triad with a target network.

In *International Conference on Machine Learning*, pages 12621–12631. PMLR.